

1. Exercise: External Memory Merge Sort

Abstract

In the world of algorithms, dealing with limited external memory brings unique challenges. Most algorithms are designed assuming a single-level memory with unlimited storage, but real-world computers organize memory hierarchically.

We implement and compare two sorting algorithms, MergeSort and External Memory MergeSort. The latter is designed to use the memory hierarchy found in a computer to sort large sequences with a minimal number of IO operations.

1 Introduction

In the world of algorithms, dealing with limited external memory brings unique challenges. Most algorithms are designed assuming a single-level memory with unlimited storage, but real-world computers organize memory hierarchically.

This study dives into External Memory MergeSort, a variation to the familiar MergeSort algorithm. Unlike its traditional version, External Memory MergeSort is built to handle the complexities of external memory, where data needs to move between different storage levels. The algorithm works in rounds, aiming to be smart about transferring data efficiently.

This paper takes a close look at External Memory MergeSort, breaking down its two main phases—Partitioning and Merging. We’ve put the algorithm to the test by implementing it in C++.

We’ll also compare it with classical MergeSort. By evaluating the practical running time and memory usage of these sorting algorithms across instances with varying size of the input based on specific proportions of the available memory, we seek to draw insightful conclusions regarding the tradeoffs and efficiency gains associated with classical and External

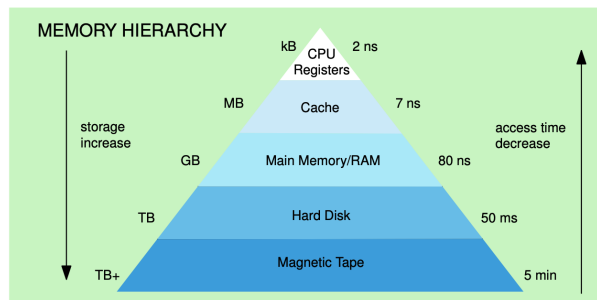


Figure 1: Memory Hierarchy, from Storandt [2023]

Memory Mergesort. We used synthetic datasets (generated randomly) to systematically test and benchmark the algorithms under various memory conditions.

2 Preliminaries

2.1 External Memory Model

The classical RAM model has two assumptions:

- There is one level of memory
- There is unlimited storage capability

But in practise, both assumptions are not met because the total memory in a computer is organized hierarchically. As displayed in figure 1, the different memory types differ in storage and access time. Data must be transferred between the different types of memory which can have a significant impact on the running time of an algorithm. To get more accurate running time predictions, the External Memory Model was developed which integrates the IO operations between two memory levels into the running time estimation of an algorithm.

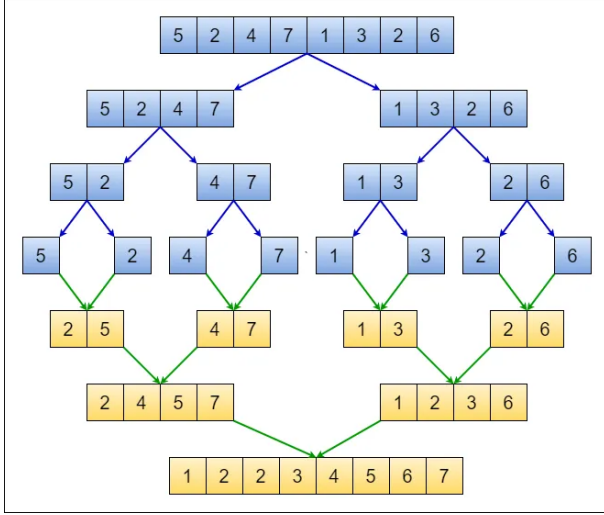


Figure 2: MergeSort Visualisation Vikram [2023]

2.2 Classical MergeSort

The MergeSort algorithm is a well-established sorting algorithm in the realm of computer science. It employs a divide-and-conquer strategy, recursively dividing the dataset into smaller chunks until individual elements are isolated. These elements are then gradually merged back together in a sorted order. In figure 2, an exemplary run of the MergeSort algorithm is visualised. The partitioning phase is colored in blue and the merging phase is colored in yellow.

3 Algorithm & Implementation

3.1 External Memory MergeSort

The External Memory MergeSort is an extension of the classical MergeSort algorithm. In this approach, sorting is performed in rounds, with a focus on minimizing the number of block transfers between internal and external memory. The algorithm uses two files, X and Y, where X points to the file containing the current input data, and Y points to an initially empty file. Sorted blocks are sequentially flushed or appended to file Y, and at the end of each round, the roles of X and Y are reversed.

The algorithm contains of 2 stages: Partitioning and Merging. The Partitioning phase aims to sort all the blocks of size M in the memory individually using a classical sorting algorithm (used classical merge sort here) as the max internal memory available is M. This is done by getting (M/B) B sized blocks the internal memory, sorting them and putting back to the external memory. The second phase that is used is the merging phase. Where we merge all (N/M) partitions of sorted memory created by the partitioning phase. This is done by choose 2 set of sorted partitions, getting B sized data from each of them, and merging them into the external memory. This same process is repeated for every 2 blocks in the partitioned data in one round. In the subsequent round, the partition size is doubled and it continues.

3.2 Implementation

The implementation of both classical MergeSort and External Memory MergeSort involves translating the algorithmic concepts into functional code. The programming language of choice, along with any external libraries, plays a crucial role in the efficiency and correctness of the implementation. For this project, we implemented the algorithms in C++, as it is fast and gives good control over memory usage. The code ensures the generation of synthetic datasets generated randomly, implementing the above-mentioned sorting algorithm, and logging of relevant metrics: time of execution and the number of rounds it took to complete the sorting.

3.3 Algorithm Engineering Concepts

There are primarily 3 algorithm engineering concepts used in the EM Mergesort algorithm:

- **External Memory Algorithms:**
The entire algorithm is designed with the consideration of external memory constraints, where data that doesn't fit into the internal memory is stored in external memory (files X and Y).
- **Optimizing I/O and Block Transfers:**
The algorithm focuses on minimizing the number of block transfers between internal and external memory.

nal memory. Sequential flushing or appending of sorted blocks to file Y is mentioned, indicating a strategy to optimize I/O operations.

- **Multi-Phase Sorting:**

The algorithm is divided into two main phases: Partitioning and Merging. This multi-phase approach helps in managing the sorting process effectively and allows for the reuse of sorted blocks in subsequent rounds.

4 Experimental Evaluation

In this section, the experimental setup is described and the results are presented.

4.1 Data and Hardware

4.1.1 Dataset Generation

To evaluate to two sorting algorithms, we first create a binary file in each iteration that contains a number of integers that have the size of the requested share of memory. These numbers range from INT_MIN(-2147483648) to INT_MAX(2147483647).

4.1.2 Hardware

The algorithms were evaluated in a docker container with a limited memory of 2GB on a host system with an Apple M2 CPU with 3.49 GHz.

4.2 Results

As we conducted the evaluation of the algorithms on a system with a total memory of 2GB, we first had to find a sensible block size parameter for that system. We chose the blocksize $B = 2^{28} = 268435456$ bytes (268,44 MB), because with $3 * B = 805,31$ MB, we can be optimistic that the memory will fit the program even if other processes allocate some memory.

We conducted the experiments with 10% and 50% of available RAM for the classical MergeSort and with 10%, 50%, 100% and 1000% of available RAM for the External Memory MergeSort. One can see the running times of classical MergeSort in table 2 and

RAM Share (%)	Block Size	Number of Rounds	Number of Integers	Running Time (s)
10	268435456	0	53687091	2.33
50	268435456	2	268435456	15.57
100	268435456	3	536870912	35.66
1000	268435456	7	5368709120	443.79

Table 1: Running Time of External Memory Merge-sort

RAM Share (%)	Number of Integers	Running Time (s)
10	53687091	2.09
50	268435456	11.43

Table 2: Running Time of Mergesort

the running times of External Memory MergeSort in table 1. For a better understanding, we plotted the results as well in figure 3 with a magnification of the smaller RAM shares in figure 4.

For the smallest data size, which is 10% of RAM, the classical MergeSort performs only about 11% slower, but for 50% of RAM the gap is already about 36%. This is explainable with the higher amount of IO operations that are performed in the EM MergeSort.

5 Discussion and Conclusion

In this work, we reviewed the MergeSort algorithm and the External Memory MergeSort algorithm, which is able to sort sequences which size is larger than the available memory. We have seen that the classical mergesort performs better for smaller input sequences but is unable to process sequences larger than RAM. The gap in the running times between the two algorithms is increasing with the input size, as the number of IO operations is increasing as well. Unless the input of the sorting algorithm does not fit into memory, the classical MergeSort is the better choice as there is less overhead.

References

S. Storandt. Lecture: 2. memory-aware algorithms, 2023.

G. Vikram. Visualizing, designing, and analyzing the merge sort algorithm. <https://medium.com/javarevisited/visualizing-designing-and-analyzing-the-merge-sort-algorithm-904ceb78a592>, 2023.

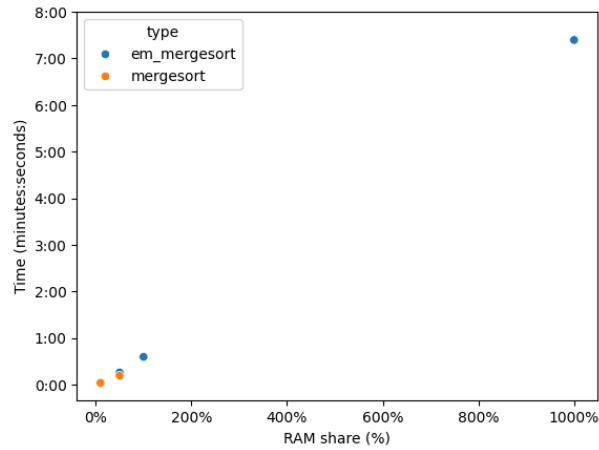


Figure 3: Running time of EM MergeSort and Merge-sort

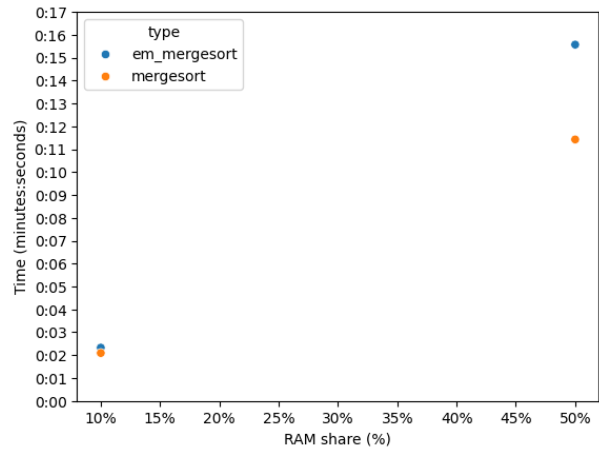


Figure 4: Magnification of figure 3